

Paradigmi di Programmazione - A.A. 2023-24

Esame Scritto del 18/06/2024

CRITERI DI VALUTAZIONE:

La prova è superata se si ottengono almeno 12 punti negli esercizi 1,2,3 e almeno 18 punti complessivamente.

Esercizio 1 [Punti 4]

Applicare la β -riduzione alla seguente λ -espressione fino a raggiungere una espressione non ulteriormente riducibile o ad accorgersi che la derivazione è infinita:

$$(\lambda g. \lambda y. g y x)(\lambda f. f y)(\lambda k. k)$$

Nella soluzione, mostrare tutti i passi di riduzione calcolati, sottolineando ad ogni passo la porzione di espressione a cui si applica la β -riduzione (redex) ed evidenziando le eventuali α -conversioni.

Esercizio 2 [Punti 4]

Indicare il tipo della seguente funzione OCaml, mostrando i passi fatti per inferirlo:

```
let f x y z =  
  if x=[] then [y]  
  else match z with  
    | [] -> (y+1)::[]  
    | z1::z' -> if z1 then [y] else [];;
```

Esercizio 3 [Punti 7]

Assumendo il seguente tipo di dato che descrive valute in Euro o Dollari:

```
type valuta =  
  | Euro of float  
  | Dollari of float;;
```

Definire in OCaml le funzioni `inEuro`, `somma_valute` e `separa_valute` con i seguenti tipi

```
inEuro : float -> valuta -> valuta  
somma_valute : float -> valuta list -> valuta  
separa_valute : valuta list -> valuta list * valuta list
```

tali che:

- `inEuro` prende un numero reale `t` che rappresenta il tasso di cambio Euro/Dollaro (attualmente 0,93 Euro per ogni Dollaro), prende `x` di tipo `valuta` e restituisce `y` di tipo `valuta`. Se `x` è una valuta espressa in Dollari, la converte in Euro moltiplicandola per `t`. Se invece è già in Euro, la restituisce così com'è.
- `somma_valute` prende un tasso di cambio `t` (come sopra) e una lista `lis` di elementi di tipo `valuta`. La funzione somma tutti gli elementi di `lis` restituendo un valore totale espresso in Euro. Eventuali valori in `lis` espressi in Dollari devono essere convertiti in Euro usando il tasso `t`.
- `separa_valute` prende una lista `lis` di elementi di tipo `valuta` e restituisce una coppia di liste (`lis1, lis2`) dove `lis1` contiene gli elementi di `lis` espressi in Euro, e `lis2` quelli espressi in Dollari.

Ad esempio,

```
inEuro 0.93 (Euro 2.0);;      (* calcola Euro 2.0 *)
inEuro 0.93 (Dollari 2.0);;    (* calcola Euro 1.86 *)
```

considerando la seguente lista di valute:

```
let lis = [Euro 1.5; Dollari (-1.0); Dollari 2.6; Euro (-2.0)];;
```

le funzioni danno i seguenti risultati:

```
somma_valute 0.93 lis;;      (* calcola Euro 0.988 *)
somma_valute 0.5 lis;;       (* calcola Euro 0.3 --- usa tasso Euro/Dollaro = 0.5 *)
separa_valute lis;;          (* calcola la coppia di liste:
                               ([Euro 1.5; Euro (-2.0)], [Dollari (-1.0); Dollari 2.6]) *)
```

Esercizio 4 [Punti 15]

Si estenda il linguaggio MiniCaml visto a lezione con un costrutto `LimFun` per definire “funzioni su interi (non ricorsive) a dominio limitato”. Tale tipologia di funzioni prevede un parametro di tipo intero, e può essere applicata solo a valori in un intervallo definito al momento della definizione della funzione stessa. Applicare la funzione a valori al di fuori di tale intervallo genera un errore.

Ad esempio (in una sintassi concreta simile a OCaml):

```
FunLim x:{1..10} -> x+1      (* definisce la funzione che incrementa di uno,
                               applicabile solo a valori tra 1 e 10 *)
```

quindi:

```
( FunLim x:{1..10} -> x+1 ) 5      (* calcola 6 *)
( FunLim x:{1..10} -> x+1 ) 0      (* errore *)
( FunLim x:{1..10} -> x+1 ) 12     (* errore *)
```

Nota: la sintassi `x:{1..10}` è inventata. E’ solo un modo per dire che il parametro si chiama `x`, il minimo valore ammissibile è 1 e il massimo è 10.

Si mostri come deve essere modificato l’interprete OCaml del linguaggio.